

CSCI 4210 — Operating Systems
Simulation Project Part II (document version 1.0)
Processes and CPU Scheduling

Overview

- This assignment is due in Submitty by 10:00PM on Wednesday, April 29, 2026
- This project is to be completed either individually or in a team of **at most three** students; form your team within the Submitty gradeable, but do **not** submit any code until we announce that auto-grading is available
- **[NEW]** If you worked on a team for Part I, feel free to change your team for Part II; if you worked alone on Part I, feel free to form a new team for Part II; all code is reusable from Part I even if you form a new team
- Beyond your team (or yourself if working alone), **do not share your code**; however, feel free to discuss the project content and your findings with one another on our Discussion Forum
- To appease Submitty, you must use one of the following programming languages: C, C++, or Python (be sure you choose only **one** language for your entire implementation)
- You will have **eight** penalty-free submissions on Submitty, after which points will slowly be deducted, e.g., -1 on submission #9, etc.
- You will have at least **three** days before the due date to submit your code to Submitty; if the auto-grading is not available three days before the due date, the due date will be 11:59PM three days after auto-grading becomes available
- You can use at most two late days on this assignment; in such cases, each team member must use a late day
- All submitted code **must** successfully compile and run on Submitty, which currently uses Ubuntu v22.04.5 LTS and gcc version 11.4.0 (Ubuntu 11.4.0-1ubuntu1~22.04.02)
- If you use C or C++, your program **must** successfully compile via gcc or g++ with no warning messages when the `-Wall` (i.e., warn all) compiler option is used; we will also use `-Werror`, which will treat all warnings as critical errors; the `-lm` flag will also be included
- For source file naming conventions, be sure to use `*.c` for C and `*.cpp` for C++; in either case, you can also include `*.h` files
- For Python, you must use `python3`, which is currently Python 3.10.12; be sure to name your main Python file `project.py`; also be sure no warning messages or extraneous output occur during interpretation
- Please “flatten” all directory structures to a single directory of source files before submitting your code (sorry)
- Note that you can use square brackets in your code

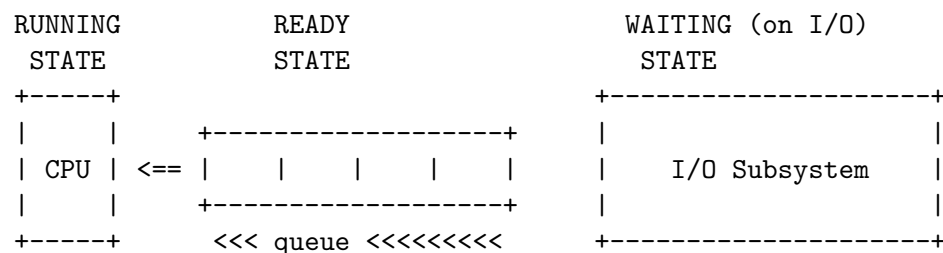
Project specifications

For our simulation project, which encompasses **both** Part I and Part II, you will implement a simulation of an operating system. The overall focus will be on processes, assumed to be resident in memory, waiting to use the CPU. Memory and the I/O subsystem will not be covered in depth.

Conceptual design

A **process** is defined as a program in execution. For this assignment, processes are in one of the following three states, corresponding to the picture shown further below.

- **RUNNING:** actively using the CPU and executing instructions
- **READY:** ready to use the CPU, i.e., ready to execute a CPU burst
- **WAITING:** blocked on I/O or some other event



Processes in the **READY** state reside in a queue called the ready queue. This queue is ordered based on a configurable CPU scheduling algorithm. There are **four** CPU scheduling algorithms that you are to implement in Part II: first-come-first-served (FCFS); shortest job first (SJF); shortest remaining time (SRT); and round robin (RR).

All implemented algorithms will be simulated for the **same set of processes**, which will therefore enable us to conduct a comparative analysis of results. In Part I, you generated useful sets of processes via pseudo-random number generators. For Part II, you will implement all four algorithms, running each are in succession with the same set of processes.

Back to the conceptual model, when a process is in the **READY** state and reaches the front of the queue, once the CPU is free to accept the next process, the given process enters the **RUNNING** state and starts executing its next CPU burst.

After each CPU burst is completed, if the process does not terminate, the process enters the **WAITING** state, waiting for an I/O (or other) operation to complete, e.g., waiting for data to be read in from a file. When the I/O operation completes, depending on the scheduling algorithm, the process either (1) returns to the **READY** state and is added to the ready queue or (2) preempts the currently running process and immediately enters the **RUNNING** state.

Note that preemptions occur only for the SRT and RR algorithms. Each algorithm is summarized on the next page.

First-come-first-served (FCFS)

The FCFS algorithm is a non-preemptive algorithm in which processes line up in the ready queue, waiting to use the CPU. This is your baseline algorithm.

Shortest job first (SJF)

In SJF, processes are stored in the ready queue in order of priority based on their anticipated CPU burst times. More specifically, the process with the shortest predicted CPU burst time will be selected as the next process executed by the CPU.

Shortest remaining time (SRT)

The SRT algorithm is a preemptive version of the SJF algorithm. In SRT, when a process arrives, as it enters the ready queue, if it has a predicted CPU burst time that is less than the remaining predicted time of the currently running process, a preemption occurs. When such a preemption occurs, the currently running process is added back to the ready queue based on its remaining predicted CPU burst time.

Round robin (RR)

The RR algorithm is essentially the FCFS algorithm with time slice t_{slice} . Each process is given t_{slice} amount of time to complete its CPU burst. If the time slice expires, the process is preempted and added to the end of the ready queue.

If a process completes its CPU burst before a time slice expiration, the next process on the ready queue is immediately context-switched in to use the CPU.

Skipping preemptions in RR

For your simulation, if a preemption occurs and there are no other processes on the ready queue, do not perform a context switch. For example, given process **G** is using the CPU and the ready queue is empty, if process **G** is preempted by a time slice expiration, do not context-switch process **G** back to the empty queue; instead, keep process **G** running with the CPU and do **not** count this as a context switch. In other words, when the time slice expires, check the queue to determine if a context switch should occur.

Simulation configuration

The key to designing a useful simulation is to provide a number of configurable parameters. This allows you to simulate and tune for a variety of scenarios, e.g., a large number of CPU-bound processes, differing average process interarrival times, multiple CPUs, etc.

Define the simulation parameters shown below as tunable constants within your code, all of which will be given as command-line arguments.

- ***(argv+1)**: Define n as the number of processes to simulate. Process IDs are assigned a two-character code consisting of an uppercase letter from A to Z followed by a number from 0 to 9. Processes are assigned in order A0, A1, A2, ..., A9, B0, B1, ..., Z9.
- ***(argv+2)**: Define n_{cpu} as the number of processes that are CPU-bound. For this project, we will classify processes as I/O-bound or CPU-bound. The n_{cpu} CPU-bound processes, when generated, will have CPU burst times that are longer by a factor of 6. Conversely, the I/O-bound processes, when generated, will have I/O burst times that are longer by a factor of 8.
- ***(argv+3)**: We will use a pseudo-random number generator to determine the *interarrival times* of CPU bursts. This command-line argument, *seed*, serves as the initial seed value for the pseudo-random number sequence. To ensure predictability and repeatability, use `srand48()` with this given seed value before simulating **each** scheduling algorithm and `drand48()` to obtain the next value in the range [0.0, 1.0).

Since Python does not have these functions, implement an equivalent 48-bit linear congruential generator, as described in the `man` page for these functions in C. Feel free to post your code for this on the Discussion Forum for others to use.

- ***(argv+4)**: To determine interarrival times, we will use an exponential distribution, as illustrated in the `exp-random.c` example. This command-line argument is parameter λ ; remember that $\frac{1}{\lambda}$ will be the average random value generated, e.g., if $\lambda = 0.01$, then the average should be approximately 100.

In the `exp-random.c` example, use the formula shown in the code, i.e., $\frac{-\ln r}{\lambda}$.

- ***(argv+5)**: For the exponential distribution, this command-line argument represents the upper bound for valid pseudo-random numbers. This threshold is used to avoid values far down the long tail of the exponential distribution. As an example, if this is set to 3000, all generated values above 3000 should be skipped. For cases in which this value is used in the ceiling function (see the next page), be sure the ceiling is still valid according to this upper bound.
- **[NEW] *(argv+6)**: Define t_{cs} as the time, in milliseconds, that it takes to perform a context switch. Specifically, the first half of the context switch time (i.e., $\frac{t_{cs}}{2}$) is the time required to remove the given process from the CPU; the second half of the context switch time is the time required to bring the next process in to use the CPU. Therefore, require t_{cs} to be a positive even integer.

- **[NEW]** `*(argv+7)`: For the SJF and SRT algorithms, since we do not know the actual CPU burst times beforehand, we will rely on estimates calculated via exponential averaging. This command-line argument is the constant α , which must be a numeric floating-point value in the range $[0, 1]$.

If this value is -1 , then your simulation does **not** perform exponential averaging and instead uses actual CPU burst times. This serves as an optimal baseline for the SJF and SRT algorithms and will be called SJF-OPT and SRT-OPT.

Note that the initial guess for each process is $\tau_0 = \frac{1}{\lambda}$.

Also, when calculating τ values, use the “ceiling” function for all calculations.

- **[NEW]** `*(argv+8)`: For the RR algorithm, define the time slice value, t_{slice} , measured in milliseconds. Require t_{slice} to be a positive integer.

Pseudo-random numbers and predictability

A key aspect of this assignment is to compare the results of each of the simulated algorithms with one another given the same initial conditions, i.e., the same initial set of processes.

To ensure each CPU scheduling algorithm runs with the same set of processes, for your Part I implementation, carefully follow the algorithm below to create a set of processes.

For each of the n processes, in order A0 through Z9, perform the steps below, with I/O-bound processes generated first. Note that all generated values are integers, even though we use a floating-point pseudo-random number generator.

Define your exponential distribution pseudo-random number generation function as `next_exp()` (or another similar name) and have it return the next pseudo-random number in the sequence, capped by `*(argv+5)`.

1. Identify the initial process arrival time as the “floor” of the next random number in the sequence given by `next_exp()`; note that you could therefore have an arrival time of 0
2. Identify the number of CPU bursts for the given process as the “ceiling” of 16 multiplied by the next random number generated from the **uniform distribution** obtained via `drand48()`; this should obtain a random integer in the inclusive range $[1, 16]$
3. For *each* CPU burst in turn, identify the CPU burst time and the I/O burst time as the “ceiling” of the next two pseudo-random numbers given by `next_exp()`; for CPU-bound processes, multiply the CPU burst time by 6; for I/O-bound processes, multiply the I/O burst time by 8 such that I/O burst time is close to an order of magnitude longer than CPU burst time; and for the last CPU burst, do not generate a corresponding I/O burst time, since each process ends with one final CPU burst

Be sure to follow the specific order of calls to `drand48()` and `next_exp()` detailed above; this will ensure that you obtain the correct sequence of pseudo-random numbers to generate your process sets.

Part II — Simulation specifics

Your simulator keeps track of elapsed time t (measured in milliseconds), which is initially 0 for each scheduling algorithm. As your simulation proceeds, t advances to each “interesting” event that occurs, displaying a specific line of output that describes each event.

The “interesting” events are:

- Start of simulation for a specific algorithm
- Process arrival (i.e., initially and at each I/O completion)
- Process starts using the CPU
- Process finishes using the CPU (i.e., completes a CPU burst)
- Process has its τ value recalculated (i.e., after a CPU burst completion)
- Process preemption (SRT and RR only)
- Process starts an I/O burst
- Process finishes an I/O burst
- Process terminates by finishing its last CPU burst
- End of simulation for a specific algorithm

Note that the “process arrival” event occurs each time a process arrives, which includes both the initial arrival time and when a process completes an I/O burst. In other words, processes “arrive” within the subsystem that consists only of the CPU and the ready queue.

The “process preemption” event occurs each time a process is preempted by a time slice expiration in RR or by an arriving process in SRT. When a preemption occurs, a context switch occurs, except when the ready queue is empty for the RR algorithm.

After you simulate each scheduling algorithm, you must reset your simulation back to the initial set of processes and set your elapsed time back to 0.

Note that there may be times during your simulation in which the simulated CPU is idle because no processes have arrived yet or all processes are busy performing I/O. Also, your simulation ends when all processes terminate.

Be sure you do not implement any additional logic for the I/O subsystem. In other words, there are no specific I/O queues to implement.

Handling “ties”

If different types of events occur at the same time, simulate these events in the following order: (a) CPU burst completion; (b) process starts using the CPU; (c) I/O burst completions; and (d) new process arrivals.

Further, any “ties” that occur *within* one of these categories are to be broken using process ID order. As an example, if processes **G1** and **C3** happen to both complete I/O bursts at the same time, process **G1** wins this “tie” (because **G1** is lexicographically before **C3**) and is therefore added to the ready queue before process **C3**.

Measurements

There are a number of measurements you will need to track in your simulation.

First, for each algorithm, count the number of preemptions and the number of context switches that occur. Further, measure CPU utilization by tracking CPU usage and CPU idle time.

Specifically, for **each CPU burst**, track CPU burst time (given), turnaround time, and wait time.

CPU burst time

CPU burst times are randomly generated for each process that you simulate via the above algorithm. CPU burst time is defined as the amount of time a process is **actually** using the CPU. Therefore, this measure does not include context switch times.

Turnaround time

Turnaround times are to be measured for each process that you simulate. Turnaround time is defined as the end-to-end time a process spends in executing a **single CPU burst**.

More specifically, this is measured from process arrival time through to when the CPU burst is completed and the process is switched out of the CPU. Therefore, this measure includes the second half of the initial context switch in and the first half of the final context switch out, as well as any other context switches that occur while the CPU burst is being completed (i.e., due to preemptions).

Wait time

Wait times are to be measured **for each CPU burst**. Wait time is defined as the amount of time a process spends waiting to use the CPU, which equates to the amount of time the given process is actually in the ready queue. Therefore, this measure does not include context switch times that the given process experiences, i.e., only measure the time the given process is actually in the ready queue.

More specifically, a process leaves the ready queue when it is switched into the CPU, which takes half of context switch time t_{cs} . Likewise, a preempted process leaves the CPU and enters the ready queue after the first half of t_{cs} .

CPU utilization

Calculate CPU utilization by tracking how much time the CPU is actively running CPU bursts versus total elapsed simulation time.

Part II — Required terminal output

Required terminal output for Part II starts by summarizing the set of generated processes (see below).

Then, your simulator displays results for each of the four algorithms you simulate. For each algorithm, display a summary for each “interesting” event that occurs from time 0 through time 9999. For time 10000 and beyond, only display process termination events, preemptions, and the final end-of-simulation event. Example output files will be posted to Submittity.

Your simulator must display a line of output for each “interesting” event that occurs using the format shown below. Note that the contents of the ready queue are shown for each event.

```
time <t>ms: <event-summary> [Q: <queue-contents>]
```

The output format must follow the examples shown below. Display λ to six digits after the decimal point; and display α to two digits after the decimal point.

In this first example, the α parameter is specified as -1 , indicating that no exponential averaging should be performed; display this simply as " α ".

```
bash$ ./a.out 3 1 32 0.001 1024 4 -1 256
<<< -- process set (n=3) with 1 CPU-bound process
<<< -- seed=32; lambda=0.001000; upper bound=1024
<<< -- t_cs=4ms; alpha=<math>\alpha</math>; t_slice=256ms
```

```
I/O-bound process A0: arrival time 319ms; 13 CPU bursts:
I/O-bound process A1: arrival time 917ms; 10 CPU bursts:
CPU-bound process A2: arrival time 229ms; 11 CPU bursts:
```

```
<<< PROJECT SIMULATIONS
```

```
time 0ms: Simulator started for FCFS [Q: -]
time 229ms: Process A2 arrived; added to ready queue [Q: A2]
time 231ms: Process A2 started using the CPU for 2814ms burst [Q: -]
time 319ms: Process A0 arrived; added to ready queue [Q: A0]
time 917ms: Process A1 arrived; added to ready queue [Q: A0 A1]
time 3045ms: Process A2 completed a CPU burst; 10 bursts to go [Q: A0 A1]
time 3045ms: Process A2 switching out of CPU; blocking on I/O until time 3554ms [Q: A0 A1]
time 3049ms: Process A0 started using the CPU for 362ms burst [Q: A1]
time 3411ms: Process A0 completed a CPU burst; 12 bursts to go [Q: A1]
time 3411ms: Process A0 switching out of CPU; blocking on I/O until time 8277ms [Q: A1]
time 3415ms: Process A1 started using the CPU for 767ms burst [Q: -]
time 3554ms: Process A2 completed I/O; added to ready queue [Q: A2]
time 4182ms: Process A1 completed a CPU burst; 9 bursts to go [Q: A2]
time 4182ms: Process A1 switching out of CPU; blocking on I/O until time 12080ms [Q: A2]
time 4186ms: Process A2 started using the CPU for 5826ms burst [Q: -]
time 8277ms: Process A0 completed I/O; added to ready queue [Q: A0]
time 31712ms: Process A2 terminated [Q: A1]
```

```

time 41684ms: Process A1 terminated [Q: A0]
time 52505ms: Process A0 terminated [Q: -]
time 52507ms: Simulator ended for FCFS [Q: -]

time 0ms: Simulator started for SJF-OPT [Q: -]
time 229ms: Process A2 arrived; added to ready queue [Q: A2]
time 231ms: Process A2 started using the CPU for 2814ms burst [Q: -]
time 319ms: Process A0 arrived; added to ready queue [Q: A0]
time 917ms: Process A1 arrived; added to ready queue [Q: A0 A1]
time 3045ms: Process A2 completed a CPU burst; 10 bursts to go [Q: A0 A1]
time 3045ms: Process A2 switching out of CPU; blocking on I/O until time 3554ms [Q: A0 A1]
time 3049ms: Process A0 started using the CPU for 362ms burst [Q: A1]
time 3411ms: Process A0 completed a CPU burst; 12 bursts to go [Q: A1]
time 3411ms: Process A0 switching out of CPU; blocking on I/O until time 8277ms [Q: A1]
time 3415ms: Process A1 started using the CPU for 767ms burst [Q: -]
time 3554ms: Process A2 completed I/O; added to ready queue [Q: A2]
time 4182ms: Process A1 completed a CPU burst; 9 bursts to go [Q: A2]
time 4182ms: Process A1 switching out of CPU; blocking on I/O until time 12080ms [Q: A2]
time 4186ms: Process A2 started using the CPU for 5826ms burst [Q: -]
time 8277ms: Process A0 completed I/O; added to ready queue [Q: A0]
time 30598ms: Process A2 terminated [Q: A1]
time 40570ms: Process A1 terminated [Q: A0]
time 51391ms: Process A0 terminated [Q: -]
time 51393ms: Simulator ended for SJF-OPT [Q: -]

time 0ms: Simulator started for SRT-OPT [Q: -]
time 229ms: Process A2 arrived; added to ready queue [Q: A2]
time 231ms: Process A2 started using the CPU for 2814ms burst [Q: -]
time 319ms: Process A0 arrived; preempting A2 (remaining time 2726ms) [Q: -]
time 323ms: Process A0 started using the CPU for 362ms burst [Q: A2]
time 685ms: Process A0 completed a CPU burst; 12 bursts to go [Q: A2]
time 685ms: Process A0 switching out of CPU; blocking on I/O until time 5551ms [Q: A2]
time 689ms: Process A2 started using the CPU for remaining 2726ms of 2814ms burst [Q: -]
time 917ms: Process A1 arrived; preempting A2 (remaining time 2498ms) [Q: -]
time 921ms: Process A1 started using the CPU for 767ms burst [Q: A2]
time 1688ms: Process A1 completed a CPU burst; 9 bursts to go [Q: A2]
time 1688ms: Process A1 switching out of CPU; blocking on I/O until time 9586ms [Q: A2]
time 1692ms: Process A2 started using the CPU for remaining 2498ms of 2814ms burst [Q: -]
time 4190ms: Process A2 completed a CPU burst; 10 bursts to go [Q: -]
time 4190ms: Process A2 switching out of CPU; blocking on I/O until time 4699ms [Q: -]
time 4699ms: Process A2 completed I/O; added to ready queue [Q: A2]
time 4701ms: Process A2 started using the CPU for 5826ms burst [Q: -]
time 5551ms: Process A0 completed I/O; preempting A2 (remaining time 4976ms) [Q: -]
time 5555ms: Process A0 started using the CPU for 79ms burst [Q: A2]
time 5634ms: Process A0 completed a CPU burst; 11 bursts to go [Q: A2]
time 5634ms: Process A0 switching out of CPU; blocking on I/O until time 9428ms [Q: A2]
time 5638ms: Process A2 started using the CPU for remaining 4976ms of 5826ms burst [Q: -]
time 9428ms: Process A0 completed I/O; preempting A2 (remaining time 1186ms) [Q: -]

```

```

time 9432ms: Process A0 started using the CPU for 889ms burst [Q: A2]
time 9586ms: Process A1 completed I/O; preempting A0 (remaining time 735ms) [Q: A2]
time 9590ms: Process A1 started using the CPU for 421ms burst [Q: A0 A2]
time 17053ms: Process A1 completed I/O; preempting A2 (remaining time 385ms) [Q: -]
time 19209ms: Process A0 completed I/O; preempting A2 (remaining time 4450ms) [Q: -]
time 20409ms: Process A1 completed I/O; preempting A2 (remaining time 3441ms) [Q: -]
time 23668ms: Process A1 completed I/O; preempting A2 (remaining time 339ms) [Q: -]
time 24964ms: Process A1 completed I/O; preempting A0 (remaining time 366ms) [Q: A2]
time 26074ms: Process A0 completed I/O; preempting A2 (remaining time 3480ms) [Q: -]
time 30223ms: Process A0 completed I/O; preempting A2 (remaining time 2588ms) [Q: -]
time 32144ms: Process A1 completed I/O; preempting A2 (remaining time 732ms) [Q: -]
time 33116ms: Process A2 completed I/O; preempting A0 (remaining time 471ms) [Q: -]
time 34103ms: Process A2 terminated [Q: A1]
time 35112ms: Process A1 terminated [Q: -]
time 41058ms: Process A0 terminated [Q: -]
time 41060ms: Simulator ended for SRT-OPT [Q: -]

time 0ms: Simulator started for RR [Q: -]
time 229ms: Process A2 arrived; added to ready queue [Q: A2]
time 231ms: Process A2 started using the CPU for 2814ms burst [Q: -]
time 319ms: Process A0 arrived; added to ready queue [Q: A0]
time 487ms: Time slice expired; preempting process A2 with 2558ms remaining [Q: A0]
time 491ms: Process A0 started using the CPU for 362ms burst [Q: A2]
time 747ms: Time slice expired; preempting process A0 with 106ms remaining [Q: A2]
time 751ms: Process A2 started using the CPU for remaining 2558ms of 2814ms burst [Q: A0]
time 917ms: Process A1 arrived; added to ready queue [Q: A0 A1]
time 1007ms: Time slice expired; preempting process A2 with 2302ms remaining [Q: A0 A1]
time 1011ms: Process A0 started using the CPU for remaining 106ms of 362ms burst [Q: A1 A2]
time 1117ms: Process A0 completed a CPU burst; 12 bursts to go [Q: A1 A2]
time 1117ms: Process A0 switching out of CPU; blocking on I/O until time 5983ms [Q: A1 A2]
time 1121ms: Process A1 started using the CPU for 767ms burst [Q: A2]
time 1377ms: Time slice expired; preempting process A1 with 511ms remaining [Q: A2]
time 1381ms: Process A2 started using the CPU for remaining 2302ms of 2814ms burst [Q: A1]
time 1637ms: Time slice expired; preempting process A2 with 2046ms remaining [Q: A1]
time 1641ms: Process A1 started using the CPU for remaining 511ms of 767ms burst [Q: A2]
time 1897ms: Time slice expired; preempting process A1 with 255ms remaining [Q: A2]
time 1901ms: Process A2 started using the CPU for remaining 2046ms of 2814ms burst [Q: A1]
time 2157ms: Time slice expired; preempting process A2 with 1790ms remaining [Q: A1]
time 2161ms: Process A1 started using the CPU for remaining 255ms of 767ms burst [Q: A2]
time 2416ms: Process A1 completed a CPU burst; 9 bursts to go [Q: A2]
time 2416ms: Process A1 switching out of CPU; blocking on I/O until time 10314ms [Q: A2]
time 2420ms: Process A2 started using the CPU for remaining 1790ms of 2814ms burst [Q: -]
time 2676ms: Time slice expired; no preemption because ready queue is empty [Q: -]
time 2932ms: Time slice expired; no preemption because ready queue is empty [Q: -]
time 3188ms: Time slice expired; no preemption because ready queue is empty [Q: -]
time 3444ms: Time slice expired; no preemption because ready queue is empty [Q: -]
time 3700ms: Time slice expired; no preemption because ready queue is empty [Q: -]
time 3956ms: Time slice expired; no preemption because ready queue is empty [Q: -]

```

time 4210ms: Process A2 completed a CPU burst; 10 bursts to go [Q: -]
 time 4210ms: Process A2 switching out of CPU; blocking on I/O until time 4719ms [Q: -]
 time 4719ms: Process A2 completed I/O; added to ready queue [Q: A2]
 time 4721ms: Process A2 started using the CPU for 5826ms burst [Q: -]
 time 4977ms: Time slice expired; no preemption because ready queue is empty [Q: -]
 time 5233ms: Time slice expired; no preemption because ready queue is empty [Q: -]
 time 5489ms: Time slice expired; no preemption because ready queue is empty [Q: -]
 time 5745ms: Time slice expired; no preemption because ready queue is empty [Q: -]
 time 5983ms: Process A0 completed I/O; added to ready queue [Q: A0]
 time 6001ms: Time slice expired; preempting process A2 with 4546ms remaining [Q: A0]
 time 6005ms: Process A0 started using the CPU for 79ms burst [Q: A2]
 time 6084ms: Process A0 completed a CPU burst; 11 bursts to go [Q: A2]
 time 6084ms: Process A0 switching out of CPU; blocking on I/O until time 9878ms [Q: A2]
 time 6088ms: Process A2 started using the CPU for remaining 4546ms of 5826ms burst [Q: -]
 time 6344ms: Time slice expired; no preemption because ready queue is empty [Q: -]
 time 6600ms: Time slice expired; no preemption because ready queue is empty [Q: -]
 time 6856ms: Time slice expired; no preemption because ready queue is empty [Q: -]
 time 7112ms: Time slice expired; no preemption because ready queue is empty [Q: -]
 time 7368ms: Time slice expired; no preemption because ready queue is empty [Q: -]
 time 7624ms: Time slice expired; no preemption because ready queue is empty [Q: -]
 time 7880ms: Time slice expired; no preemption because ready queue is empty [Q: -]
 time 8136ms: Time slice expired; no preemption because ready queue is empty [Q: -]
 time 8392ms: Time slice expired; no preemption because ready queue is empty [Q: -]
 time 8648ms: Time slice expired; no preemption because ready queue is empty [Q: -]
 time 8904ms: Time slice expired; no preemption because ready queue is empty [Q: -]
 time 9160ms: Time slice expired; no preemption because ready queue is empty [Q: -]
 time 9416ms: Time slice expired; no preemption because ready queue is empty [Q: -]
 time 9672ms: Time slice expired; no preemption because ready queue is empty [Q: -]
 time 9878ms: Process A0 completed I/O; added to ready queue [Q: A0]
 time 9928ms: Time slice expired; preempting process A2 with 706ms remaining [Q: A0]
 time 9932ms: Process A0 started using the CPU for 889ms burst [Q: A2]
 time 10188ms: Time slice expired; preempting process A0 with 633ms remaining [Q: A2]
 time 10448ms: Time slice expired; preempting process A2 with 450ms remaining [Q: A0 A1]
 time 10708ms: Time slice expired; preempting process A0 with 377ms remaining [Q: A1 A2]
 time 10968ms: Time slice expired; preempting process A1 with 165ms remaining [Q: A2 A0]
 time 11228ms: Time slice expired; preempting process A2 with 194ms remaining [Q: A0 A1]
 time 11488ms: Time slice expired; preempting process A0 with 121ms remaining [Q: A1 A2]
 time 12496ms: Time slice expired; preempting process A2 with 262ms remaining [Q: A1]
 time 12756ms: Time slice expired; preempting process A1 with 505ms remaining [Q: A2]
 time 13016ms: Time slice expired; preempting process A2 with 6ms remaining [Q: A1]
 time 13276ms: Time slice expired; preempting process A1 with 249ms remaining [Q: A2]
 time 19132ms: Time slice expired; preempting process A2 with 3536ms remaining [Q: A1]
 time 19392ms: Time slice expired; preempting process A1 with 14ms remaining [Q: A2]
 time 19652ms: Time slice expired; preempting process A2 with 3280ms remaining [Q: A1]
 time 19930ms: Time slice expired; preempting process A2 with 3024ms remaining [Q: A0]
 time 20190ms: Time slice expired; preempting process A0 with 373ms remaining [Q: A2]
 time 20450ms: Time slice expired; preempting process A2 with 2768ms remaining [Q: A0]
 time 20710ms: Time slice expired; preempting process A0 with 117ms remaining [Q: A2]

time 20970ms: Time slice expired; preempting process A2 with 2512ms remaining [Q: A0]
time 21351ms: Time slice expired; preempting process A2 with 2256ms remaining [Q: A0]
time 22822ms: Time slice expired; preempting process A2 with 976ms remaining [Q: A1]
time 26159ms: Time slice expired; preempting process A2 with 2342ms remaining [Q: A1]
time 26763ms: Time slice expired; preempting process A2 with 1830ms remaining [Q: A1]
time 27023ms: Time slice expired; preempting process A1 with 394ms remaining [Q: A2 A0]
time 27283ms: Time slice expired; preempting process A2 with 1574ms remaining [Q: A0 A1]
time 27543ms: Time slice expired; preempting process A0 with 212ms remaining [Q: A1 A2]
time 27803ms: Time slice expired; preempting process A1 with 138ms remaining [Q: A2 A0]
time 28063ms: Time slice expired; preempting process A2 with 1318ms remaining [Q: A0 A1]
time 28681ms: Time slice expired; preempting process A2 with 1062ms remaining [Q: A1]
time 29023ms: Time slice expired; preempting process A2 with 806ms remaining [Q: A0]
time 34218ms: Process A2 terminated [Q: -]
time 38416ms: Time slice expired; preempting process A1 with 237ms remaining [Q: A0]
time 38676ms: Time slice expired; preempting process A0 with 589ms remaining [Q: A1]
time 38917ms: Process A1 terminated [Q: A0]
time 44342ms: Process A0 terminated [Q: -]
time 44344ms: Simulator ended for RR [Q: -]

In this second example, α is set to 0.75. The output below is abbreviated for the FCFS and RR algorithms, since the output for each will match the previous example above.

```
bash$ ./a.out 3 1 32 0.001 1024 4 0.75 256
<<< -- process set (n=3) with 1 CPU-bound process
<<< -- seed=32; lambda=0.001000; upper bound=1024
<<< -- t_cs=4ms; alpha=0.75; t_slice=256ms
```

```
I/O-bound process A0: arrival time 319ms; 13 CPU bursts
I/O-bound process A1: arrival time 917ms; 10 CPU bursts
CPU-bound process A2: arrival time 229ms; 11 CPU bursts
```

<<< PROJECT SIMULATIONS

```
time 0ms: Simulator started for FCFS [Q: -]
...
time 52507ms: Simulator ended for FCFS [Q: -]
```

```
time 0ms: Simulator started for SJF [Q: -]
time 229ms: Process A2 (tau 1000ms) arrived; added to ready queue [Q: A2]
time 231ms: Process A2 (tau 1000ms) started using the CPU for 2814ms burst [Q: -]
time 319ms: Process A0 (tau 1000ms) arrived; added to ready queue [Q: A0]
time 917ms: Process A1 (tau 1000ms) arrived; added to ready queue [Q: A0 A1]
time 3045ms: Process A2 (tau 1000ms) completed a CPU burst; 10 bursts to go [Q: A0 A1]
time 3045ms: Recalculated tau for process A2 to 2361ms [Q: A0 A1]
time 3045ms: Process A2 switching out of CPU; blocking on I/O until time 3554ms [Q: A0 A1]
time 3049ms: Process A0 (tau 1000ms) started using the CPU for 362ms burst [Q: A1]
time 3411ms: Process A0 (tau 1000ms) completed a CPU burst; 12 bursts to go [Q: A1]
time 3411ms: Recalculated tau for process A0 to 522ms [Q: A1]
time 3411ms: Process A0 switching out of CPU; blocking on I/O until time 8277ms [Q: A1]
time 3415ms: Process A1 (tau 1000ms) started using the CPU for 767ms burst [Q: -]
time 3554ms: Process A2 (tau 2361ms) completed I/O; added to ready queue [Q: A2]
time 4182ms: Process A1 (tau 1000ms) completed a CPU burst; 9 bursts to go [Q: A2]
time 4182ms: Recalculated tau for process A1 to 826ms [Q: A2]
time 4182ms: Process A1 switching out of CPU; blocking on I/O until time 12080ms [Q: A2]
time 4186ms: Process A2 (tau 2361ms) started using the CPU for 5826ms burst [Q: -]
time 8277ms: Process A0 (tau 522ms) completed I/O; added to ready queue [Q: A0]
time 31420ms: Process A2 terminated [Q: A1]
time 41392ms: Process A1 terminated [Q: A0]
time 52213ms: Process A0 terminated [Q: -]
time 52215ms: Simulator ended for SJF [Q: -]
```

```
time 0ms: Simulator started for SRT [Q: -]
time 229ms: Process A2 (tau 1000ms) arrived; added to ready queue [Q: A2]
time 231ms: Process A2 (tau 1000ms) started using the CPU for 2814ms burst [Q: -]
time 319ms: Process A0 (tau 1000ms) arrived; added to ready queue [Q: A0]
time 917ms: Process A1 (tau 1000ms) arrived; added to ready queue [Q: A0 A1]
time 3045ms: Process A2 (tau 1000ms) completed a CPU burst; 10 bursts to go [Q: A0 A1]
```

```

time 3045ms: Recalculated tau for process A2 to 2361ms [Q: A0 A1]
time 3045ms: Process A2 switching out of CPU; blocking on I/O until time 3554ms [Q: A0 A1]
time 3049ms: Process A0 (tau 1000ms) started using the CPU for 362ms burst [Q: A1]
time 3411ms: Process A0 (tau 1000ms) completed a CPU burst; 12 bursts to go [Q: A1]
time 3411ms: Recalculated tau for process A0 to 522ms [Q: A1]
time 3411ms: Process A0 switching out of CPU; blocking on I/O until time 8277ms [Q: A1]
time 3415ms: Process A1 (tau 1000ms) started using the CPU for 767ms burst [Q: -]
time 3554ms: Process A2 (tau 2361ms) completed I/O; added to ready queue [Q: A2]
time 4182ms: Process A1 (tau 1000ms) completed a CPU burst; 9 bursts to go [Q: A2]
time 4182ms: Recalculated tau for process A1 to 826ms [Q: A2]
time 4182ms: Process A1 switching out of CPU; blocking on I/O until time 12080ms [Q: A2]
time 4186ms: Process A2 (tau 2361ms) started using the CPU for 5826ms burst [Q: -]
time 8277ms: Process A0 (tau 522ms) completed I/O; added to ready queue [Q: A0]
time 13270ms: Process A1 (tau 523ms) completed I/O; preempting A2
               (predicted remaining time 624ms) [Q: -]
time 22642ms: Process A0 (tau 715ms) completed I/O; preempting A2
               (predicted remaining time 3438ms) [Q: -]
time 23389ms: Process A0 (tau 651ms) completed I/O; preempting A2
               (predicted remaining time 3328ms) [Q: -]
time 25416ms: Process A1 (tau 378ms) completed I/O; preempting A2
               (predicted remaining time 1492ms) [Q: -]
time 28675ms: Process A1 (tau 207ms) completed I/O; preempting A2
               (predicted remaining time 3105ms) [Q: -]
time 28930ms: Process A0 (tau 300ms) completed I/O; preempting A2
               (predicted remaining time 2942ms) [Q: -]
time 30060ms: Process A0 (tau 426ms) completed I/O; preempting A2
               (predicted remaining time 2942ms) [Q: -]
time 33228ms: Process A2 terminated [Q: -]
time 39358ms: Process A1 (tau 153ms) completed I/O; preempting A0
               (predicted remaining time 415ms) [Q: -]
time 40367ms: Process A1 terminated [Q: A0]
time 46041ms: Process A0 terminated [Q: -]
time 46043ms: Simulator ended for SRT [Q: -]

time 0ms: Simulator started for RR [Q: -]
...
time 44344ms: Simulator ended for RR [Q: -]

```

Part II — Required output file

In addition to the above output (which is sent to `stdout`), generate an output file called `simout.txt` that contains some general statistics followed by specific statistics for each simulated algorithm. The file format is shown below, with `#` as a placeholder for actual numerical data. If calculating an average will cause division by 0, simply set the average to 0.

```
-- number of processes: #
-- number of CPU-bound processes: #
-- number of I/O-bound processes: #
-- CPU-bound average CPU burst time: ### ms
-- I/O-bound average CPU burst time: ### ms
-- overall average CPU burst time: ### ms
-- CPU-bound average I/O burst time: ### ms
-- I/O-bound average I/O burst time: ### ms
-- overall average I/O burst time: ### ms
```

Algorithm <algorithm>

```
-- CPU utilization: ###%
-- CPU-bound average wait time: ### ms
-- I/O-bound average wait time: ### ms
-- overall average wait time: ### ms
-- CPU-bound average turnaround time: ### ms
-- I/O-bound average turnaround time: ### ms
-- overall average turnaround time: ### ms
-- CPU-bound number of context switches: #
-- I/O-bound number of context switches: #
-- overall number of context switches: #
-- CPU-bound number of preemptions: #
-- I/O-bound number of preemptions: #
-- overall number of preemptions: #
```

For the RR algorithm, also include the following (appended to the end):

```
-- CPU-bound percentage of CPU bursts completed within one time slice: ###%
-- I/O-bound percentage of CPU bursts completed within one time slice: ###%
-- overall percentage of CPU bursts completed within one time slice: ###%
```


Continuing the first example from page 9, the `simout.txt` file is as shown below. A set of full `simout.txt` output files will be posted on Submittity.

```
-- number of processes: 3
-- number of CPU-bound processes: 1
-- number of I/O-bound processes: 2
-- CPU-bound average CPU burst time: 2226.55 ms
-- I/O-bound average CPU burst time: 410.26 ms
-- overall average CPU burst time: 997.88 ms
-- CPU-bound average I/O burst time: 335.40 ms
-- I/O-bound average I/O burst time: 3072.00 ms
-- overall average I/O burst time: 2189.23 ms
```

Algorithm FCFS

```
-- CPU utilization: 64.62%
-- CPU-bound average wait time: 326.82 ms
-- I/O-bound average wait time: 822.48 ms
-- overall average wait time: 662.12 ms
-- CPU-bound average turnaround time: 2557.36 ms
-- I/O-bound average turnaround time: 1236.74 ms
-- overall average turnaround time: 1664.00 ms
-- CPU-bound number of context switches: 11
-- I/O-bound number of context switches: 23
-- overall number of context switches: 34
-- CPU-bound number of preemptions: 0
-- I/O-bound number of preemptions: 0
-- overall number of preemptions: 0
```

Algorithm SJF-OPT

```
-- CPU utilization: 66.02%
-- CPU-bound average wait time: 225.55 ms
-- I/O-bound average wait time: 725.61 ms
-- overall average wait time: 563.82 ms
-- CPU-bound average turnaround time: 2456.09 ms
-- I/O-bound average turnaround time: 1139.87 ms
-- overall average turnaround time: 1565.71 ms
-- CPU-bound number of context switches: 11
-- I/O-bound number of context switches: 23
-- overall number of context switches: 34
-- CPU-bound number of preemptions: 0
-- I/O-bound number of preemptions: 0
-- overall number of preemptions: 0
```

Algorithm SRT-OPT

```
-- CPU utilization: 82.63%
-- CPU-bound average wait time: 540.18 ms
-- I/O-bound average wait time: 38.52 ms
-- overall average wait time: 200.82 ms
```

-- CPU-bound average turnaround time: 2774.73 ms
-- I/O-bound average turnaround time: 453.30 ms
-- overall average turnaround time: 1204.35 ms
-- CPU-bound number of context switches: 22
-- I/O-bound number of context switches: 26
-- overall number of context switches: 48
-- CPU-bound number of preemptions: 11
-- I/O-bound number of preemptions: 3
-- overall number of preemptions: 14

Algorithm RR

-- CPU utilization: 76.51%
-- CPU-bound average wait time: 546.27 ms
-- I/O-bound average wait time: 344.30 ms
-- overall average wait time: 409.65 ms
-- CPU-bound average turnaround time: 2785.18 ms
-- I/O-bound average turnaround time: 761.52 ms
-- overall average turnaround time: 1416.24 ms
-- CPU-bound number of context switches: 34
-- I/O-bound number of context switches: 40
-- overall number of context switches: 74
-- CPU-bound number of preemptions: 23
-- I/O-bound number of preemptions: 17
-- overall number of preemptions: 40
-- CPU-bound percentage of CPU bursts completed within one time slice: 18.18%
-- I/O-bound percentage of CPU bursts completed within one time slice: 43.48%
-- overall percentage of CPU bursts completed within one time slice: 35.29%

Error handling

If improper command-line arguments are given, report an error message to `stderr` and abort further program execution. Be sure that the number of processes is greater than 0; further, deduce other valid input value ranges from the specifications given on page 4.

In general, if an error is encountered, display a meaningful error message on `stderr`, then abort further program execution.

Error messages must be one line only and use the following format:

```
ERROR: <error-text-here>
```

Submission instructions

To submit your assignment and also perform final testing of your code, please use Submittity.

Please be sure to perform adequate testing on your own Ubuntu platform.

Relinquishing allocated resources

Be sure that all resources (e.g., dynamically allocated memory) are properly relinquished for whatever language/platform you use for this assignment. Sloppy programming will likely result in lower grades. Consider doing frequent code reviews with your teammates if working on a team.

OPTIONAL — Analysis questions to think about...

As you work on this project, consider the questions below. Though there is no formal write-up required for this project, you should be able to answer at least some of these questions based on specific results from your simulation. A few of these questions would require features beyond the given requirements.

1. Of the four simulated algorithms, which algorithm is the “best” algorithm for CPU-bound processes? Which algorithm is best-suited for I/O-bound processes?
2. For the SJF and SRT algorithms, what value of α produced the “best” results? How close to optimal could you get with different values of α ?
3. For the SJF and SRT algorithms, how does changing from a non-preemptive algorithm to a preemptive algorithm impact your results? Are there specific examples of processes (or types of processes) that have improved turnaround or wait times?
4. Identify at least three limitations of your simulation, in particular how the project specifications could be expanded to better model a real-world operating system.
5. Identify other “interesting” events and measurements that you could incorporate into your simulation.
6. Describe a priority scheduling algorithm of your own design, i.e., how could you calculate priority? What are advantages and disadvantages of your algorithm?

Again, the above questions are included here for you to think about and discuss with your teammates and/or others in the course. Feel free to open a discussion about these on our Discussion Forum.